EXERCISE 1: SEMEVAL 2026 - TASK 13A BINARY MACHINEGENERATED CODE DETECTION

Nguyen Truong Thinh - 25C11066
[1]Faculty of Information Technology
University of Science – VNU-HCM
Ho Chi Minh City, Vietnam

Abstract

This paper describes our system submitted to SemEval-2026 Task 13 on binary classification of code snippets as human-written or AI-generated. The training data consists of roughly 500K algorithmic (LeetCode-style) samples in C++, Python, and Java, while the 50K-sample test set is out-of-distribution, spanning unseen languages (Go, PHP, C#, C, JavaScript) and unseen domains (research and production code). We combine 30 handcrafted stylistic features with perplexity signals from a pretrained code language model, perform ensemble feature selection via XGBoost and LightGBM, explore Isolation Forest for robust outlier-based detection, and try to fine-tune CodeBERT as a neural baseline. The final stacked ensemble proves robust across all OOD regimes.

Source code. Source code is available at https://github.com/ThinhSama234/SemEval/tree/main.

1 Introduction

The rapid adoption of large language models (LLMs) such as GPT-4, Claude, and Code LLaMA has transformed software development, enabling developers to generate functional code from natural-language prompts with minimal effort. While this capability accelerates productivity, it also raises pressing concerns around academic integrity, authorship attribution, and the reliability of automated code review. Consequently, the task of distinguishing **human-written code** from **machine-generated code** has emerged as a critical research problem, and is the focus of SemEval-2026 Task 13 (kaggle challenge).

Limitations of supervised learning under distribution shift. A fundamental limitation of conventional supervised detectors is their inability to generalize beyond the distribution observed during training. Models trained on a fixed set of programming languages and problem domains tend to exploit surface-level stylistic cues indentation patterns, identifier naming conventions, or boilerplate structures that are specific to the training distribution. When deployed on *out-of-distribution* (OOD) inputs, such as unseen programming languages (Go, PHP, C#, C, JavaScript) or unseen domains (research and production code), performance degrades sharply. This challenge is amplified in SemEval-2026 Task 13, where the evaluation is explicitly stratified into seen, partially unseen, and fully unseen regimes, exposing the brittleness of detectors that rely on memorized lexical patterns.

Recent approaches and pretrained foundations. Recent work on machine-generated text detection has followed two complementary directions. The first leverages **pretrained code language models** such as CodeBERT, GraphCodeBERT, CodeT5, and UniXcoder, which encode transferable syntactic and semantic representations across programming languages. Fine-tuning these encoders has become a standard baseline for downstream classification tasks. The second direction exploits *intrinsic statistical signals* produced by generative models themselves most notably token-level *perplexity*, log-likelihood curvature (DetectGPT), and rank-based measures (GPTZero, Ghostbuster) which capture the characteristic low-entropy distributions of LLM outputs. However, these model-based signals are computationally expensive to extract and difficult to deploy under constrained inference budgets. In contrast, handcrafted stylometric features including lexical attributes like comment density and indentation regularity, as well as structural properties derived from **Abstract Syntax Trees** (ASTs) remain competitive, particularly in low-resource and cross-lingual settings where neural encoders suffer from domain mismatch.

Our proposal. Motivated by the OOD nature of SemEval-2026 Task 13, we argue that no single family of methods is sufficient: neural encoders

offer contextual depth but may overfit to seen languages, while statistical features generalize but lack semantic sensitivity. We therefore propose a hybrid system built around three stages. First, we extract 20 language-agnostic, handcrafted style features covering indentation regularity, comment patterns, token entropy, naming consistency, and structural complexity organized into five semantic groups (entropy/complexity, comment/documentation, structure, surface, and a global group) (section §4.1). Second, we apply a multi-group Isolation Forest strategy: for each feature group we train two Isolation Forest models, one fitted on AI-generated samples and one on human-written samples, producing 15 anomaly scores per snippet that capture how "out-of-distribution" each sample appears relative to each class. These scores are fed into a Complement Naïve Bayes classifier whose decision threshold is optimized on the validation set for macro F1 (section §4.2). We additionally explore gradient-boosted ensembles (XGBoost, LightGBM, CatBoost) as alternative classifiers, and a stacking layer that combines the above with frozen CodeBERT embeddings (section §4.3, §4.4 §4.5). A different approach, Perplexity-based features from a pretrained code language model were investigated but excluded from the final system due to prohibitive inference cost and limited generalization to unseen languages.

2 Related Work

The detection of machine-generated text has become increasingly important as large language models grow more capable. We review three lines of work most relevant to our approach.

Machine-generated text detection. Early methods relied on statistical artifacts visible to simpler language models. Gehrmann et al. (2019) proposed GLTR, which visualizes token-level likelihood under GPT-2 to help human annotators spot generated text. More recently, Mitchell et al. (2023) introduced DetectGPT, exploiting the observation that LLM-generated passages tend to lie in regions of negative log-probability curvature, enabling zero-shot detection without fine-tuning. Watermarking-based approaches (Kirchenbauer et al., 2023) embed imperceptible statistical signals during generation, but require access to the generation process. Supervised detectors such as Ghostbuster (Verma et al., 2024) combine features from multiple weaker language models to achieve robust detection without relying on a single reference model. However, most of these methods target natural language prose and do not account for the structural properties unique to source code.

Pretrained code models. A parallel line of research has developed pretrained representations specifically for programming languages. CodeBERT (Feng et al., 2020) applies masked language modeling and replaced token detection over six languages, producing cross-lingual code embeddings. GraphCodeBERT (Guo et al., 2021) extends this by incorporating data flow graphs, while UniXcoder (Guo et al., 2022) unifies code understanding and generation in a single encoder-decoder framework. These models have been successfully fine-tuned for downstream tasks including code search (Feng et al., 2020), code summarization (Ahmad et al., 2020), and vulnerability detection (Zhou et al., 2019). Their cross-lingual pretraining makes them natural candidates for OOD generalization to unseen programming languages.

Ensemble and anomaly-based methods. Gradient-boosted tree ensembles such as XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017) remain strong baselines for tabular classification tasks, including stylometric authorship analysis, and have been shown to consistently outperform deep learning on structured feature representations (Shwartz-Ziv and Armon, 2022). Stacking (Wolpert, 1992) combines heterogeneous base learners through a meta-classifier and has been shown to improve robustness when individual models capture complementary signals. Anomaly detection methods, particularly Isolation Forest (Liu et al., 2008), offer an alternative paradigm that models class-specific density rather than learning a direct decision boundary—a property that may improve robustness under distribution shift. The idea of using unsupervised outlier scores as engineered features for a supervised classifier is explored in XGBOD (Zhao and Hryniewicki, 2019), which demonstrates that Isolation Forest anomaly scores passed as input features to XGBoost consistently improve over either method alone. Our work extends this direction by combining anomaly-score features from group-wise Isolation Forests with gradient-boosted classifiers and, as a stacking layer, frozen representations from a pretrained code encoder.

3 Dataset

3.1 Overview

SemEval-2026 Task 13A provides a large-scale dataset for binary classification of code snippets as human-written (label 0) or AI-generated (label 1). The training set contains approximately 500,000 labeled samples: 238K human-written (47.7%) and 262K machine-generated (52.3%). A separate validation set of 100K samples follows the same distribution. The unlabeled test set comprises 500,000 samples targeting out-of-distribution generalization.

3.2 Languages and Domains

All training and validation samples are restricted to three programming languages **C++**, **Python**, and **Java** drawn from **algorithmic problem-solving** contexts (e.g., competitive programming, LeetCode-style problems). This design explicitly tests the generalization capacity of detection systems beyond their training distribution.

3.3 AI Generators

The machine-generated portion of the dataset is produced by 35 distinct code generation models spanning a wide range of architectures and parameter scales, including CodeLlama (7B–70B), StarCoder, DeepSeek-Coder (1.3B–33B), Phi-3, Yi-Coder, and Granite. This diversity ensures that detectors cannot overfit to the output style of a single generator family.

4 Proposed Methodology

In this section, we describe our three-stage methodology. First, we perform exploratory data analysis (EDA) to engineer discriminative features spanning stylistic patterns and syntactic tree structure. Second, we employ Isolation Forest as an anomaly-based detector to identify outliers that deviate from learned class-specific patterns. Finally, we fine-tune CodeBERT a pretrained code language model as the backbone of our system, leveraging its cross-lingual code representations to generalize beyond seen languages and domains.

4.1 Handcrafted Feature Extraction

Stylistic features. With the support of AI-assisted analysis, we evaluate distributional differences between human-written and machine-generated code and design 24 language-agnostic features organized into four groups: *lexical*, *formatting*, *naming & structure*, and *documentation & consistency*. The core intuition behind our feature design stems from a simple observation: **humans and machines write code differently, not just in logic, but in style**. A human developer writes code under time pressure, shaped by personal habits inconsistent spacing, terse variable names, few comments, and occasional formatting shortcuts. In contrast, an LLM generates code by sampling from a learned distribution, producing output that is statistically "too clean": overly structured, richly commented, and stylistically uniform. Our features are designed to capture exactly these behavioral differences across the following four dimensions:

- **Lexical statistics (5 features):** Shannon entropy of character distribution (`token_entropy`), type token ratio (`ttr`), keyword density (`keyword_density`), duplicate line ratio (`duplicate_line_ratio`), and token bigram repetition rate (`bigram_repetition`). These features capture how predictable and repetitive the code is AI-generated code tends to be more uniform and compressible, exhibiting lower entropy and higher repetition.

- **Formatting patterns (6 features):** Standard deviation of indentation levels (`indentation_std`), fraction of empty lines (`blank_line_ratio`), standard deviation of line lengths (`line_len_std`), line-length burstiness (`line_len_burstiness`), trailing whitespace ratio (`trailing_ws_ratio`), and mean blank lines per function (`blank_per_function`). AI models tend to produce overly regular formatting consistent indentation and predictable blank-line patterns whereas human code varies with personal habits and editing history.

- **Naming and structural features (7 features):** camelCase vs. snake_case uniformity (`style_consistency`), dominant naming convention frequency (`naming_uniformity`), maximum bracket nesting depth (`max_nesting_depth`), mean lines per indented block (`avg_block_length`), count of branching keywords as a cyclomatic complexity proxy (`cyclomatic_proxy`), function size regularity (`function_size_regularity`),

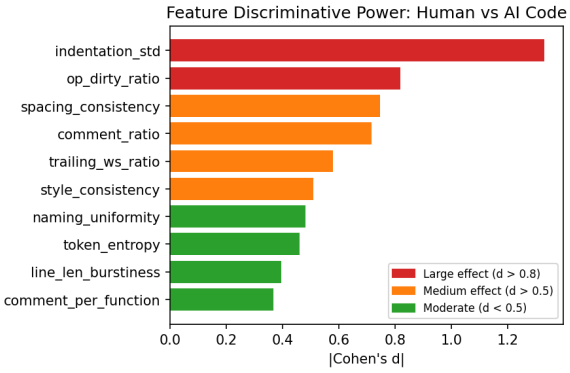

Figure 1: Top 10 discriminative features from our 24-feature set, ranked by |Cohen's d| effect size. Red: large effect ($d > 0.8$), orange: medium ($d > 0.5$), green: moderate ($d < 0.5$). Tab/space indicators are excluded despite high effect sizes due to language-dependent bias.

and mean comment lines per function (`comment_per_function`). AI-generated code favors longer, more descriptive identifiers and consistent naming conventions, while human developers tend to use shorter, ad hoc identifiers.

- **Documentation and operator consistency (6 features):** Comment-to-code ratio (`comment_ratio`), inline comment ratio (`inline_comment_ratio`), fraction of functions preceded by a docstring or comment (`comment_completeness`), assignment operator spacing inconsistency (`eq_dirty_ratio`), general operator spacing inconsistency (`op_dirty_ratio`), and overall spacing uniformity (`spacing_consistency`). AI models produce significantly more comments $12.2\times$ the density of human code on our training split and apply operator spacing far more consistently, resulting in near-zero dirty ratios. (image 1)

AST-derived structural features. To complement surface-level stylistic signals with deeper syntactic information, we extract 10 tree-structure features from each code snippet. For Python snippets, we parse the exact abstract syntax tree using the built-in `ast` module and perform a breadth-first traversal to compute node-level statistics. For all other languages where a universal parser would require heavy runtime dependencies we apply a language-agnostic bracket-depth heuristic that approximates structural metrics using bracket nesting and token analysis. The 10 features are:

- **Tree depth:** maximum nesting depth (`ast_max_depth`), mean depth across all nodes (`ast_mean_depth`), and depth variance (`ast_depth_variance`). The depth variance is particularly informative: AI-generated code tends to exhibit more uniform nesting structure, whereas human-written code shows irregular depth variation reflecting ad hoc control flow.

- **Tree shape:** total node count (`ast_node_count`), leaf-to-node ratio (`ast_leaf_ratio`), and average branching factor (`ast_avg_branching`). These features capture how dense and balanced the syntax tree is AI-generated code often produces larger, more balanced trees due to its tendency toward complete and structured implementations.

- **Node type distribution:** Shannon entropy over node type frequencies (`ast_type_entropy`), capturing the diversity of syntactic constructs used. High entropy indicates varied code structure, while low entropy suggests repetitive or templated patterns common in LLM output.

- **Function signatures:** number of function definitions (`ast_func_count`), mean argument count (`ast_func_arg_mean`), and its standard deviation (`ast_func_arg_std`). AI-generated code tends to define more functions with more consistent argument counts, reflecting its bias toward modular, well-structured code.

Combined with the 24 stylistic features, our full handcrafted feature set contains **34 features** per snippet. This richer representation provides both surface-level style signals and tree-structural signals, enabling the downstream classifier to distinguish human from AI code at multiple levels of abstraction.

However, the results did not meet our expectations. Although the model achieved strong accuracy on the in-distribution training and validation sets, performance dropped significantly on the test set. The root cause is the out-of-distribution nature of the evaluation: unseen programming languages (Go, PHP, C, C, JavaScript) carry entirely different formatting conventions, syntactic idioms, and boilerplate structures, rendering many of our carefully

engineered features ineffective. Features that reliably separate human from AI code in Python such as indentation patterns or comment density may behave very differently in C or JavaScript, where language conventions, IDE defaults, and community style guides introduce confounding variation. This limitation motivated us to explore methods that operate on deeper code representations rather than surface-level style alone.

4.2 Isolation Forest–Based Anomaly Transformation

Anomaly-score transformation via Isolation Forest. Rather than learning a direct decision boundary over raw style features, we transform each sample into a structured anomaly-score representation. The 20 stylistic features are first partitioned into four semantically coherent groups—*entropy & complexity*, *comment & documentation*, *structure*, and *surface*—plus one group containing all 20 features, yielding $K = 5$ groups in total.

For each group k, features are normalised with a `QuantileTransformer` (uniform output distribution) to remove scale sensitivity. Two Isolation Forests are then trained separately: $\mathrm{IF}_{\mathrm{ai}}^{(k)}$ fitted exclusively on AI-generated training samples, and $\mathrm{IF}_{\mathrm{hum}}^{(k)}$ fitted exclusively on human-written samples. At inference time, each group produces three scalar scores via the Isolation Forest decision function:

$$s_{\mathrm{ai}}^{(k)}, \quad s_{\mathrm{hum}}^{(k)}, \quad s_{\mathrm{diff}}^{(k)} = s_{\mathrm{ai}}^{(k)} - s_{\mathrm{hum}}^{(k)}, \qquad (1)$$

where higher values indicate greater typicality within the respective class distribution. Concatenating across all K groups yields a 15-dimensional anomaly-score vector per sample (3×5), which is then shifted to a non-negative domain and passed to a Complement Naïve Bayes classifier (Rennie et al., 2003) to produce $P(\mathrm{AI} \mid \mathbf{s})$.

This transformation is *nonlinear*, *distribution-free*, and captures class-conditional density information that linear projections discard. The decision threshold is calibrated by grid search over $[0.20, 0.80]$ on the validation set to maximise macro F1. Key hyperparameters—contamination rate $\in \{0.02, 0.05, 0.10, 0.15\}$, number of trees $\in \{150, 300\}$, and CNB smoothing $\alpha \in \{0.5, 1.0, 2.0\}$—are selected via the same validation criterion, yielding 24 configurations evaluated in total.

4.3 Gradient-Boosted Ensemble

We train a three-model soft-voting ensemble combining XGBoost, LightGBM, and CatBoost on the full handcrafted feature set. All three models share the same core hyperparameters: 2,000 estimators, learning rate 0.03, maximum tree depth 7, subsampling ratio 0.8, column sampling 0.7, ℓ_1 penalty $\alpha = 0.5$, and ℓ_2 penalty $\lambda = 3.0$, with early stopping after 100 non-improving rounds evaluated on the validation set. We additionally append four interaction features (entropy $\times$ compression ratio, tokens per line, comment density, and line-length coefficient of variation) and a heuristic language identifier, yielding a combined input space of 76 features. Perplexity computed from a pretrained code language model is included as an additional feature for the training set; validation and test samples are filled with the training-set median, as inference-time perplexity computation was found to be prohibitively expensive.

Ensemble weights are optimised by exhaustive grid search over $\{0.5, 1.0, 1.5, 2.0, 2.5\}^3$ on the validation set, with the decision threshold jointly tuned over $[0.20, 0.80]$ to maximise macro F1.

We validate cross-generator robustness via Leave-One-Generator-Out (LOGO) cross-validation across 34 AI generator families. A lighter XGBoost model (300 estimators, learning rate 0.1, max depth 6) is retrained on all generators except the held-out one and evaluated on held-out AI samples paired with all human samples. This yields a mean macro F1 of 0.949 ($\sigma = 0.014$), with the weakest generalisation on `deepseek-ai/deepseek-coder-1.3b-instruct` (F1 = 0.909) and the strongest on `microsoft/Phi-3-medium-4k-instruct` (F1 = 0.977).

4.4 Multi-Model Soft-Voting Ensemble

The three gradient-boosted classifiers are combined via weighted soft voting: final probabilities are computed as the weighted average of per-model softmax outputs, with weights selected by grid search. While the ensemble reduced variance across generators, it yielded only marginal gains over a single well-tuned XGBoost. This result suggests that diversity in *feature representation*—not model architecture—is the primary bottleneck: combining three models over the same feature space adds redundancy rather than complementary information, motivating our shift toward

neural representations in the next stage.

4.5 Fine-tuning CodeBert

Full fine-tuning. In the first strategy, we fine-tune the entire CodeBERT model for binary sequence classification. Code snippets are tokenised with truncation to 512 tokens; due to compute constraints on Kaggle P100 GPUs, we use a maximum sequence length of 256 and subsample up to 100,000 training examples per run. Training proceeds for 3 epochs with a batch size of 32 and gradient accumulation over 2 steps (effective batch size 64), AdamW optimiser (learning rate 2×10^{-5}, weight decay 0.01), a linear warmup over 10% of total steps, and FP16 mixed precision. We evaluate macro F1 on a 10,000- sample validation subset every third of an epoch, retaining the best checkpoint via early stopping (patience 3).

To encourage the model to focus on hard-to-classify samples— particularly those from programming languages underrepresented in training— we experiment with focal loss (Lin et al., 2017):

$$\mathcal{L}_{\text{focal}} = -\alpha\,(1 - p_t)^{\gamma}\,\log p_t, \qquad (2)$$

with $\gamma = 2$ and optional label smoothing ($\epsilon = 0.1$). We further experiment with an active-learning loop: an initial seed of 50,000 samples is used for a fast warm-up epoch, then the remaining pool is scored by prediction uncertainty ($|p - 0.5|$), the 50,000 most uncertain samples are added, and training continues for 2 additional rounds.

Frozen embeddings with lightweight head. In the second strategy, CodeBERT weights are frozen entirely. We extract the `[CLS]` token embedding (dimension 768) from the last hidden state for each snippet, then concatenate it with the 20 language-agnostic style features described in Section §4.1 (normalised via `QuantileTransformer` to a standard normal distribution), yielding a 788-dimensional input vector.

A lightweight MLP head is then trained on this concatenated. Training uses AdamW (lr 10^{-3}, weight decay 10^{-4}) with cosine annealing over up to 50 epochs and early stopping (patience 5) on validation macro F1. This strategy avoids the risk of catastrophic forgetting and substantially reduces compute: embeddings are extracted once and cached, and only the 788-dimensional head is trained. The combination of contextual semantic representations with interpretable style features provides complementary signals that neither source alone captures.

5 Experiments and results

Our submitted system is a three-way soft-voting ensemble combining (i) a language-robust gradient-boosted ensemble on 75 handcrafted features, (ii) a fine-tuned CodeBERT with a calibrated decision threshold $\tau = 0.93$, and (iii) an IsolationForest + ComplementNB hybrid trained on 20 style features. Raw probabilities from each constituent are rank-normalised to $[0, 1]$, averaged, and thresholded so that the predicted AI ratio matches the training distribution (52%). This system scored macro F1 = 0.527 on the hidden test set.

Validation–test gap. Validation F1 is strongly anti-correlated with leaderboard performance. Our strongest individual model on validation (the 83-feature gradient-boosted ensemble) reached val F1 = 0.989 but only 0.254 on the hidden test — a gap of 0.735. Conversely, the comparatively weak IsolationForest + ComplementNB baseline (val F1 = 0.823) scored 0.535 on the hidden test, outperforming every more sophisticated single model we trained – and, marginally, the final soft-voting ensemble itself (0.527).

Feature-shift diagnosis. We computed the standardised mean difference $|d|$ between train and test feature distributions. The most over-shifted features are all identifier- and syntax-related: `avg_identifier_len` ($|d|$=1.76), `id_avg_len` (1.69), `id_short_ratio` (1.54), `line_entropy_std` (1.40), `camel_case_ratio` (1.26), `burstiness` (1.15), `punct_density` (1.05). By contrast, whitespace-related features show near-zero shift: `tab_count` ($|d|$=0.000), `leading_tab_lines` (0.029), `comment_ratio` (0.010). We attribute the shift to the presence of unseen brace-heavy programming languages in the test set — JavaScript, TypeScript, Go, Ruby, C# — whose naming conventions and syntax density are under-represented in our training corpus of 91.5% Python, 4.7% C++ and 3.9% Java.

Mitigation. Removing the seven features with $|d|$>1.0, together with perplexity (which is prohibitively expensive to compute on the 500 K-sample test set), reduces the feature set from 83 to 75 and yields our language-robust ensemble. This raises test F1 by 0.218 (0.254 → 0.472) at a cost of 0.016 on validation (0.989 → 0.973).

System	#Feat	Val F1	Test F1
Single classifiers on handcrafted features			
XGBoost	76	0.982	–
LightGBM	76	0.984	–
CatBoost	76	0.981	–
Constituents of the submitted ensemble			
Gradient-Boosted Ensemble (original)	83	0.989	0.254
Gradient-Boosted Ensemble (language-robust)	75	0.973	0.472
CodeBERT (fine-tuned, τ=0.93)	–	0.954	0.451
IsolationForest + ComplementNB	20	0.823	**0.535**
Combined systems			
LR Stacking (lang-robust + IF+CNB)	–	0.990	0.359
Soft-Voting (lang-robust + CodeBERT + IF+CNB)	–	0.986	**0.527**

Table 1: Macro F1 on the 100K-sample validation set and the hidden test set. The competition publishes a single overall test score, so per-regime columns (Seen/Unseen combinations of language and generator family) remain unfilled. Bold: our final submitted system (Soft-Voting) and its strongest single constituent (IF + CNB).

Ensemble effects. Rank-average soft voting of the three constituents improves test F1 by 0.055 over the best handcrafted backbone (0.472 $\rightarrow$ 0.527). The improvement does not extend over the IF + CNB constituent (0.535 $\rightarrow$ 0.527, -0.008): blending dilutes its strong test signal, and we retain the ensemble for its lower variance across language regimes. A logistic-regression stacking meta-learner trained on validation probabilities scored substantially lower at 0.359: because the language-robust ensemble was early-stopped on the validation set, its validation probabilities are nearly perfect (val F1 $= 0.973$), which led the meta-learner to assign it a dominant coefficient ($\approx$10.9) and collapse the ensemble's diversity.

Cross-generator robustness. Table 3 reports leave-one-generator-out cross-validation of a lighter XGBoost model across 34 AI generator families. Mean macro F1 is 0.949 with $\sigma = 0.014$. The weakest generalisation is to `deepseek-coder-1.3b-instruct` (F1 $= 0.909$) and the strongest to `Phi-3-medium-4k-instruct` (F1 $= 0.977$). Since generator-shift degradation is bounded by $\sim$0.04, the 0.5–0.7 gap observed between validation and hidden test is attributable to language shift, not generator shift.

Metric	Value
Generators evaluated	34
Mean macro F1	0.949
Std (σ)	0.014
Min F1 (`deepseek-1.3b-it`)	0.909
Max F1 (`Phi-3-medium`)	0.977

Table 2: Leave-One-Generator-Out cross-validation of the lighter XGBoost model across 34 AI generator families.

6 Conclusion

We presented a systematic exploration of handcrafted-feature and neural approaches for AI code detection on SemEval 2026 Task A. Despite strong in-domain validation performance (macro F1 $>$ 0.98 for our ensembles), the hidden test set exposed a substantial out-of-distribution shift. Our final three-way soft-voting system reached 0.527, which — notably — does not exceed the simple IsolationForest + ComplementNB baseline at 0.535.

Our diagnostic analysis isolates programming-*language* shift, rather than *generator* shift, as the dominant source of the generalisation gap: identifier-naming and syntax features show train–test $|d|>1.0$ while indentation and comment features remain stable. Leave-one-generator-out cross-validation separately confirms that the detector generalises well across unseen generators (σ=0.014).

Three lessons emerge. (1) Validation F1 is an unreliable proxy for the hidden test set when training data covers a narrow set of programming languages; future systems should incorporate a held-out language as an additional selection criterion. (2) Prediction-distribution balance is a strong indicator of hidden-test performance: every submission whose predicted AI ratio exceeded 80% scored below 0.40. (3) Learned stacking meta-learners can *hurt* ensembles when constituent probabilities are contaminated by early-stopping on the validation set; simple rank-averaging proved more robust.

We release our code, training pipeline, and feature-shift diagnostic to support follow-up work

Metric	Value
Generators evaluated	34
Mean macro F1	0.949
Std (σ)	0.014
Min F1 (`deepseek-1.3b-it`)	0.909
Max F1 (`Phi-3-medium`)	0.977

Table 3: Leave-One-Generator-Out cross-validation of the lighter XGBoost model across 34 AI generator families.

on language-robust AI-code detection.

7 Limitations

Our work has several limitations that should be acknowledged:

- **Out-of-distribution generalization.** Despite efforts to design language-agnostic features, our system was developed and validated exclusively on C++, Python, and Java. Stylistic conventions in unseen languages (Go, PHP, C#, C, JavaScript) and unseen domains (research and production code) may differ in ways that our features do not capture. The true OOD performance remains uncertain until test set results are released.

- **Inference cost.** Computing perplexity requires a full forward pass through a 1.5B-parameter model (failed), and CodeBERT fine-tuning adds further computational overhead. This limits the scalability of our system to large-scale, real-time deployment scenarios compared to lightweight feature-only approaches.

- **Adversarial robustness.** Our features rely on stylistic patterns (formatting, naming, comment density) that can be easily manipulated by post-processing or adversarial paraphrasing. A determined adversary could normalize indentation, strip comments, or rename variables to evade detection.

- **Binary framing.** The task assumes a strict human-vs-machine dichotomy, but real-world code increasingly involves human–AI collaboration (e.g., AI-assisted autocompletion, human-edited AI drafts). Our system is not designed to handle such mixed-authorship scenarios.

References

Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A transformer-based approach for source code summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4998–5007.

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547.

Sebastian Gehrmann, Hendrik Strobelt, and Alexander Rush. 2019. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 111–116.

Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7212–7225.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *ICCV*.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422.

Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Jason D M Rennie, Lawrence Shih, Jaime Teevan, and David R Karger. 2003. Tackling the poor assumptions of naive bayes text classifiers. In *ICML*.

Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90.

Vivek Verma, Eve Fleisig, Nicholas Tomlin, and Dan Roth. 2024. Ghostbuster: Detecting text ghostwritten by large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

David H. Wolpert. 1992. Stacked generalization. *Neural Networks*, 5(2):241–259.

Yue Zhao and Maciej K Hryniewicki. 2019. XGBOD: Improving supervised outlier detection with unsupervised representation learning. *arXiv preprint arXiv:1912.00290*.

Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32.

A CodeBERT Architecture and Fine-Tuning Details

A.1 Background

CodeBERT (Feng et al., 2020) is a bimodal pretrained model built on the RoBERTa architecture (12 Transformer layers, 768 hidden dimensions, 12 attention heads, 125M parameters). It is pretrained on a large-scale corpus of 6.4 million bimodal data points (code–documentation pairs) and 6.1 million unimodal code samples across six programming languages: Python, Java, JavaScript, PHP, Ruby, and Go. The pretraining objective combines:

- **Masked Language Modeling (MLM):** 15% of input tokens are randomly masked, and the model learns to predict them from context enabling it to capture syntactic and semantic patterns of code.
- **Replaced Token Detection (RTD):** A generator produces plausible substitutions for masked tokens, and the model learns to distinguish original tokens from generated replacements a signal directly relevant to our task of detecting machine-generated code.

A.2 Why CodeBERT for This Task

CodeBERT is particularly suited to SemEval-2026 Task 13 for three reasons:

1. **Cross-lingual pretraining.** The model was pretrained on six languages, including JavaScript, PHP, and Go three of the five unseen test languages. This provides a natural inductive bias for OOD generalization that handcrafted features lack.
2. **Contextual representations.** Unlike our 30 handcrafted features that capture surface-level statistics, CodeBERT produces 768-dimensional contextual embeddings that encode token-level semantics, control flow patterns, and idiomatic usage subtle signals that may differ between human and AI authorship.
3. **RTD pretraining signal.** The replaced token detection objective is conceptually aligned with our task: during pretraining, CodeBERT already learned to distinguish "natural" tokens from "machine-generated" replacements.

A.3 CodeBERT Fine-Tuning

Table 4 reports the hyperparameters used for both full fine-tuning (§4.5) and the frozen-embedding variant.

Hyperparameter	**Value**
Full fine-tuning	
Base model	`microsoft/codebert-base` (125M)
Max sequence length	256 tokens
Train / val samples	100K / 10K (stratified)
Epochs	3
Batch size	32×2 grad-accum (eff. 64)
Learning rate / warmup	2×10^{-5} / 0.1
Weight decay	0.01
Precision	FP16 (mixed)
Loss	Focal (γ=2) + LS (ϵ=0.1)
Early stopping	patience 3 (macro F1)
Active learning (optional)	
Seed / query / rounds	50K / 50K / 3
Query strategy	Least confidence ($\lvert p - 0.5\rvert$)
Frozen CodeBERT + MLP head	
Input	768 (`[CLS]`) + 20 style = 788
Architecture	$788 \to 256 \to 64 \to 2$, BN + Dropout (0.3, 0.2)
Optimiser / LR	AdamW / 10^{-3}, cosine ($T_{\max}$=30)
Weight decay	10^{-4}
Batch / epochs	2,048 / 50 (early stop pat. 5)

Table 4: Fine-tuning and frozen-embedding configurations for CodeBERT.